University of Carthage
Higher School of Communications of Tunis
SUPCOM

Graduation Project

Submitted for the degree of:

NATIONAL TELECOMMUNICATION ENGINEERING DEGREE

Field: Telecommunication

Major: Communicating System and Connected Objects (SCOC)

Design and development of an intelligent recommender system in e-commerce

---

Prepared at: Telnet Holding

Achieved by:
Manel ALOUI

Supervised by:
Dr. Manel BEN ROMDHANE (SUPCOM)
Ms. Nouha KAMOUN (Telnet)

Annual university:
2019/2020

“You can’t wait for customers to come to you. You have to figure out where they are, go there and drag them back to your store.”- Paul Graham, Y COMBINATOR"

"A lot of times, people don’t know what they want until you show it to them. -Steve Jobs”

Acknowledgements

This project has lasted four months in order to obtain the national degree of engineering in Telecommunication by the Higher School of Communication of Tunis; SUPCOM. And it has been done in collaboration with Telnet Holding Lac 2.

Every success hides behind it great people who are always so supportive.That is why I reserve this page as a sign of gratitude and a sign of recognition to all those without whom this project would not be realised.

I would like to express my deepest thanks not only to my supervisor Nouha KAMOUN but also to all the team, for the interest which they granted me throughout this project and for the precious knowledge which they kindly wanted to share with me.

My absolute gratitude to my academic advisor Dr. Manel BEN ROMDHANE for her availability and for the valuable advice that she offered to me. I thank her for the kindness and generosity with which she has directed this work.

I would also like to thank my beloved family for their continuous encouragement, support and help in every step that I take. They give me the strength that I need to go forward. I would also like to thank a number of friends that were particularly supportive along this path.
All my gratitude to all my teachers for their and treasurable training during my study years.

Abstract

Recommender systems are one of the most successful and widespread application of machine learning technologies in business. They are used to predict the "rating" or "preference" that a user would give to an item. Almost every e-commerce company has applied them in some form or another: like for instance Amazon that uses it to suggest products to customers. There are many approaches to tackle this topic relying on different data sets varying from matrix factorization to natural language processing in order to recommend relevant items.
Thus to fix the adopted approach we design the whole system relying on the input data that identify each user.

So in this project we study theory behind a recommender system, its algorithms and we develop one in e-commerce field especially in Fashion domain.

keywords: *Recommender system, Collaborative filtering, Matrix factorization, Natural language processing*

Contents

Acknowledgements ii

Abstract iii

List of Figures vi

List of Tables vii

Abbreviations viii

Introduction 1

1 Recommender system: algorithms and approaches 3
1.1 Introduction . . . 3
1.2 Overview of recommender system . . . 3
1.2.1 How does a recommender system work: . . . 3
1.2.2 Required Characteristics of a recommender system: . . . 4
1.3 Recommender systems techniques . . . 4
1.3.1 Content-based filtering: . . . 4
1.3.2 Collaborative filtering . . . 6
1.3.2.1 Memory-based approaches: . . . 7
1.3.2.2 Model-based approaches . . . 7
1.3.3 Hybrid filtering . . . 8
1.4 Matrix factorization: . . . 9
1.5 Natural Language Processing in RecSys: . . . 11
1.5.1 Definition: . . . 11
1.5.1.1 Artificial intelligence: . . . 11
1.5.1.2 Natural Language Processing : . . . 11
1.5.1.3 Sentiment Analysis . . . 12
1.5.2 Text Classification: . . . 12
1.5.2.1 Naive Bayes . . . 12
1.5.2.2 Logistic regression . . . 13
1.6 Goals and objectives . . . 14
1.7 Conclusion . . . 14

2 Methodology 15
2.1 Introduction . . . 15

2.2 Pipeline of the project . . . 15
2.2.1 Data-set description . . . 16
2.2.2 Data pre-processing . . . 17
2.2.3 Exploratory data analysis . . . 19
2.3 Rating Prediction based on Matrix Factorization . . . 21
2.3.1 From utility matrix to cost function . . . 21
2.3.2 Learning algorithm: Stochastic Gradient Descent . . . 23
2.3.3 Implementation process . . . 24
2.4 Rating prediction based on text classification using Natural language Processing . . . 25
2.4.1 Text pre processing . . . 27
2.4.2 Feature selection . . . 28
2.4.3 Classification . . . 29
2.5 Conclusion . . . 29

3 Experimental results and discussions **30**
3.1 Introduction . . . 30
3.2 Implementation tools . . . 30
3.2.1 Google colab . . . 30
3.2.2 Surprise A Python scikit for recommender systems . . . 31
3.2.3 NLTK . . . 31
3.3 Results and discussions . . . 32
3.3.1 Matrix factorization results and discussion . . . 32
3.3.1.1 Evaluation metrics for MF . . . 32
3.3.2 Matrix factorization results and discussion . . . 32
3.3.3 Sentiment analysis results and discussion . . . 36
3.3.3.1 Evaluation metrics for classification . . . 36
3.3.3.2 Results of text classification . . . 37
3.4 Conclusion . . . 39

Conclusion and Future Perspectives **39**

Bibliography **41**

List of Figures

1.1 Content-based filtering example . . . 5
1.2 Collaborative filtering example . . . 6
1.3 Hybrid filtering RecSys . . . 8
1.4 Classification of RecSys Design Techniques . . . 9
1.5 Matrix factorization illustration . . . 10
1.6 How AI and NLP work together . . . 12
1.7 Sigmoid function . . . 13

2.1 Pipeline of the project . . . 16
2.2 Data set overview . . . 17
2.3 Rating distribution . . . 19
2.4 Number of rating per month . . . 20
2.5 Probability Density Function . . . 20
2.6 Utility matrix illustration . . . 21
2.7 Latent matrix . . . 22
2.8 Pipeline of the implementation process . . . 24
2.9 Why we used the sentiment analysis? . . . 26
2.10 Text classification process . . . 26
2.11 Text pre-processing example . . . 28

3.1 Google Colaboratory logo . . . 30
3.2 Python package logo : Surprise . . . 31
3.3 RMSE train data with different latent factors . . . 33
3.4 RMSE test data with different latent factors . . . 33
3.5 Training time . . . 34
3.6 Variation of RMSE with several λ values . . . 34
3.7 Variation of RMSE with several α values . . . 35
3.8 Confusion matrix . . . 36
3.9 Confusion matrix results . . . 38

List of Tables

1.1 A comparatif table of CF techniques 8

2.1 Comparatif study of the data-set 17
2.2 MF hyper parameters grid search 25

3.1 Result table 38

Abbreviations

AI	**A**rtificial **I**ntelligence
CBF	**C**ontent **B**ased **F**iltring
CF	**C**ollaborative **F**iltring
ML	**M**achine **L**earning
LFM	**L**atent **F**actor **M**odel
MF	**M**atrix **F**actorization
NLP	**N**atural **L**anguange **P**rocessing

Introduction

E-commerce, also known as electronic commerce or internet commerce, refers to the buying and selling of goods or services using the internet [1].
In the last years e-commerce has been growing, in fact the total value of e-commerce in Africa reached $16.5 billion in 2017 and is expected to reach $29 billion by 2022 [2]. Thus the main goal is to stay competitive and meet the variety of special needs and tastes of a worldwide market.
This has important consequences for consumers both positives and negatives customer, in fact they are inundated with choices. Therefore, although a product or service might exist, sometimes it can be really hard to find it.
Consequently, companies have tried to predict, segment and personalize the available offer by exploiting massive datasets.Thus matching consumers with the most appropriate products is a key factor to enhance the user loyalty and satisfaction, and this is where recommender systems comes into the picture. So recommender systems are an Artificial Intelligence AI technology that has became an essential part of business for many e-commerce sites such Amazon which 35% of its revenue is generated by its recommender system.
Reports said that recommender system Market size is projected to reach $12.03 billion by 2025 from $1.14 billion in 2018, with a Compound Annual Growth Rate CAGR [1] of 32.36% between 2020 and 2025.[2]
And it is within the overall framework of this idea that Telnet Holding wants to invest in recommender systems RecSys via Machine learning Ml.
This intern is part of a project developed by Telnet Holding titled Smartecom which aims to create an intelligent recommender system for e-commerce websites and a chatbot who can at a later stage maintain a discussion between them.
My task is to design and develop an intelligent recommender system that propose items to users.
The outline of this report is as follows:

[1] CAGR: Compound Annual Growth Rate; is the mean annual growth rate of an investment over a specified period of time longer than one year

- **Recommender system: algorithms and approaches** This part consists of introducing the importance of this project in its environment and focusing on the preliminary study of recommender system by detailing its approaches in order to end up later with the adopted one.
- **Methodology:** In this stage, we focus on the workflow and the architecture followed during the model building.
- **Experimental results and discussions:** This chapter presents the different experimental results of our work with some brief discussions and comments.
- **Conclusion and perspectives:** This is the synthesis of all our efforts with some perspectives.

Chapter 1

Recommender system: algorithms and approaches

1.1 Introduction

In this first chapter the project context and the need of RecSys are presented. Starting by introducing the types of a RecSys then the problem statement and the goals.
Afterwards, we will shed the light on AI and spatially machine learning and Natural language processing. This Chapter is finally concluded by an in-depth explanation of some text classification algorithms.

1.2 Overview of recommender system

Interests in recommender system (RecSys) design started in 1990's when the first works to improve a user's experience were proposed [3]. Over the past decade, RecSys have become an important component of e-commerce platformes, and are widely employed in areas like movie/music recommendations[4], book recommendations[5], fashion industries [6] and so on.

1.2.1 How does a recommender system work:

Recommender systems are based on understanding an individual user's behaviour and then recommending to them products they might find useful.
The recommendation problem may be formulated as a rating prediction problem;it means we predict the rating that a user u gave to an item i, here we speak about a user-item combination.
It supposes that we already have training data which indicate user preferences for items. For

m users and n items, this corresponds to an incomplete m × n matrix, where the specified (or observed) values are used for training. The missing (or unobserved) values are predicted using this training model. Thus this problem is also referred to a matrix completion problem since we have incompletely specified matrix of values, and the remaining values are predicted by the learning algorithm.

1.2.2 Required Characteristics of a recommender system:

Recommendation systems are intelligent tools used to generate relevant suggestions for the users. There are some subjective characteristics specific to the business area and behind this there are some general features of a good RecSys discussed below.

- **Quality of Prediction:** This means generating meaningful recommendations that are aligned with a user's preference so that a customer's interest in the portal is maintained.

- **Speed of Computation:** An effective RecSys should be equipped to handle many registered customers and an equally large number of items then generate relevant suggestions in reasonable time, so the speed of processing is a major criterion for any RecSys design algorithm.

- **Applicability of Design:** This property details the manner to use the available information in order to fulfill the above requirement.

1.3 Recommender systems techniques

Recommender systems use a number of different technique which are classified into three main groups.

- **Content-based filtering**
- **Collaborative filtering**
- **Hybrid filtering**

1.3.1 Content-based filtering:

The content-based filtering algorithm propose products which are similar to the ones that one user has liked in the past.
So, the idea here is to tag products using certain keywords, understand what the user likes, look up those keywords in the database and recommend different products with the same

attributes. Here we mean by attribute for example genre or artist for movies and size or color for clothes[7]. In the figure 1.1 we see a detailed example on content-based recommender system.

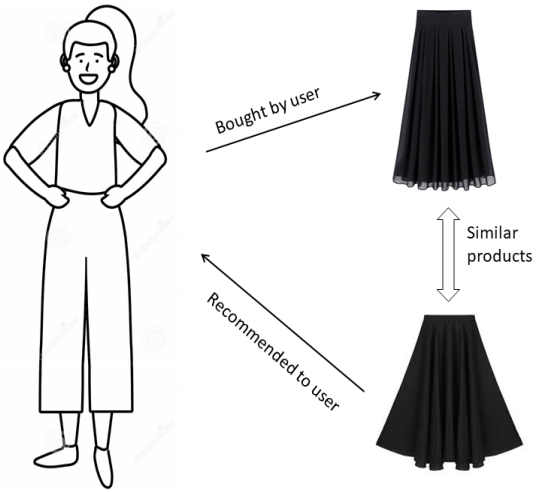


FIGURE 1.1: Content-based filtering example

To recap basically we build the profile vector A which contains the user behaviour; his rating given to items and an item vector B in which we save each feature that can describe this item example color, size, brand etc.. and then we compute the similarity between the A and B using for example the cosine similarity metric given in the next equation:

$$sim(A,B) = \cos(A,B) = \frac{A.B}{\left\|\overrightarrow{A}\right\| \left\|\overrightarrow{B}\right\|} \tag{1.1}$$

Based on the similarity function value, which ranges between -1 to 1, the products are sorted in descending order and one of the two below approaches can be used for recommendations:

- Top-n approach: where the top n items are recommended (Here n can be decided by the business owner)
- Rating scale approach: Where a threshold is fixed and all the items above that threshold are recommended.

The main drawback of content based filtering CBF is that it is limited to recommending products that are of the same type it means products having same characteristics [2] (features). It will never recommend products which the user has not bought or liked previously. So if a user has bought or liked only sports clothing in the past, the system will recommend only that type of clothing like sports shoes, sports cap ect.. and it will not propose another type

like evening dresses or skirts.
Also CBF are not good at capturing inter-dependencies or complex behaviors so it's a very narrow way of building a recommender system.
To improve on this kind of system, we need an another algorithm that can recommend items not just based on the content (description), but the behavior of users as well.

1.3.2 Collaborative filtering

This algorithm recommends items based on similarity measures between users and/or items. The items recommended to a user are those preferred by similar users.
Here we use the known interactions between users and items to predict relevant interactions/preferences for the future. The interaction between users and items is captured via the rating matrix named also utility matrix (having users as rows and items as columns).
The rating can be in the range from 1 to 5, or it can be binary 1 or 0, which is decided by the specific business (the plateform).
So here basically we focus on the user-item matrix to deduce about the correlation between them and based on that the system recommend items to other like-minded users.
In the figure 1.2 we see an example of collaborative filtering system.

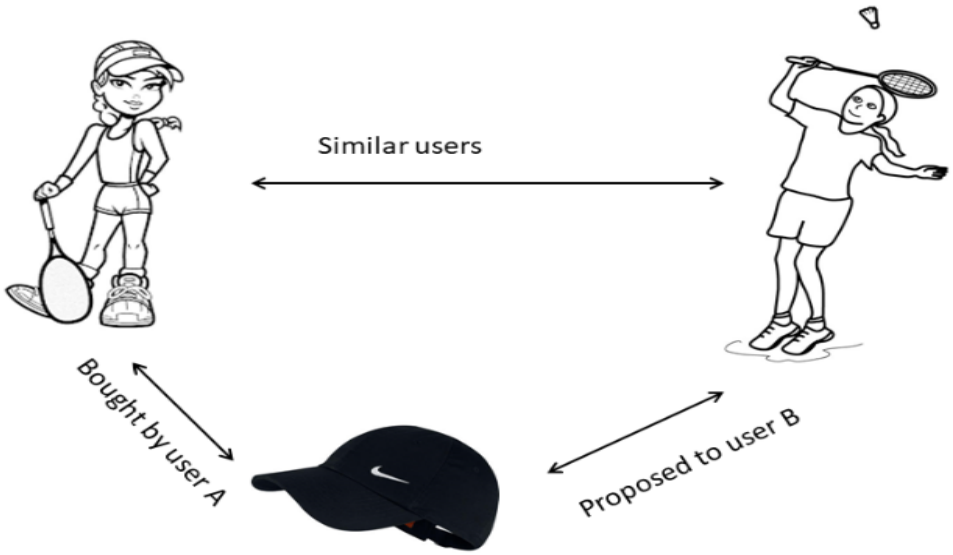


FIGURE 1.2: Collaborative filtering example

The collaborative filtering algorithm can be further divided into two categories: memory-based collaborative filtering called also (neighborhood-based collaborative filtering) and model-based collaborative filtering.

1.3.2.1 Memory-based approaches:

The main idea in memory-based methods is to use either user-user similarity or item-item similarity to make recommendations from a ratings matrix, so we need to determine either similar users or similar items in order to make predictions.
There are two basic principles used in memory-based models:

- User-based models: Also called user-user models in which similar users have similar ratings on the same item Therefore, if we have two users user A and user B that have rated products in a similar way in the past, then one can use user A's observed ratings on one unrated product by user B to predict it.
- Item-based models: Also called item-item models in which similar items are rated in a similar way by the same user. Therefore, user A's ratings on similar type products like sport clothing can be used to predict his rating on a brand of caps for example.
  And for both methods correlation computation is done using a similarity function like cosine similarity.

Memory-based models are intuitive designs thus, easy to understand and interpret. However, the need to scan the entire rating matrix (more than several thousand users and items) for the task of rating prediction makes memory based models computationally slower.

1.3.2.2 Model-based approaches

In Model-based also, called user-item models, we find Machine Learning (ML) [1] and data mining [2]methods are used in the context of predictive models. In cases where the model is parameterized, the parameters of this model are learned within the context of an optimization framework[7]
And this is also known as latent factor models which basically rely on identifying latent (hidden) factor (features) that affect the user's preferences.
In content-based filtering approach, we feed user's history behaviour and explicitly defined features to map all products and users to those features space. But in latent factor model (LFM), we only feed user's history without descriptors or factors. The algorithm will find the hidden factors that impact the user's preference like brand of product, price of product in clothing industry and so on.
The main idea of LFM is to think of recommendation as an optimization problem it means we think of it as rating prediction so the optimization here is to find the method that gives

[1] Machine Learning is defined as the field of computer science that make the computer able to take decisions without being explicitly programmed.This is done by training the computer to find out relations inside the data.
[2] Data mining is the process of finding anomalies, patterns and correlations within large data sets to predict outcomes.

us the best rating prediction with the minimum error; the smaller this rating is the better the recommendation.
In the table 1.1 we summarize CF techniques with its differences.

TABLE 1.1: A comparatif table of CF techniques

Approach	Definition	Advantages	Disadvantages
Memory-based	Find similar users based on cosine similarity and take weighted average of ratings.	Easy creation and explainability of results.	Performance reduces when data is sparse, so no scalable.
Model-based	Use machine learning to find user ratings of unrated items. e.g Matrix Factorization.	Dimentionality reduction deals with missing/ sparse data.	Inference is intracable because of hidden/latent factors.

1.3.3 Hybrid filtering

The hybrid RecSys approach combine both CF and CBF approaches as shown in 1.3.

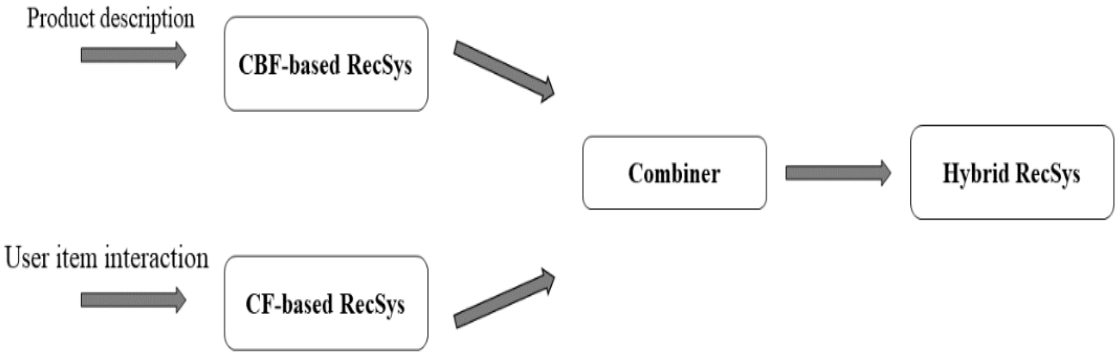


FIGURE 1.3: Hybrid filtering RecSys

Hybrid approaches can be implemented in several ways; by making CBF and CF predictions separately and then combining them, or by adding CBF capabilities to a CB approach (and vice versa), or by unifying the two approaches into one model.
It's basically constructed by using the user-item rating and the item description [8]. Research have proven that this technique is power full In the figure above we resume all the kinds of RecSys with some advantages and disadvantages.

In figure 1.4 we introduce the RecSys techniques as well as a comparative study.

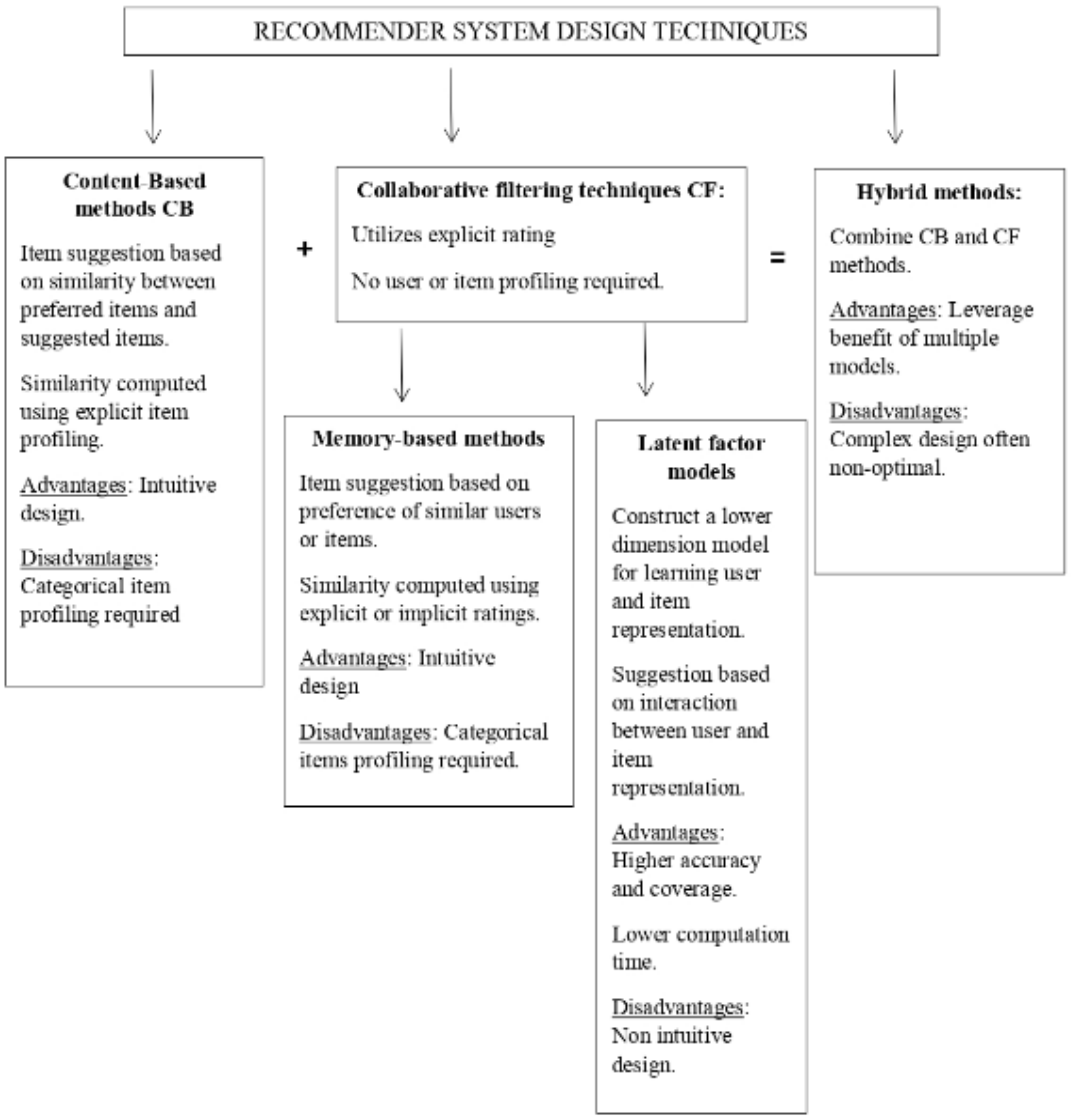


FIGURE 1.4: Classification of RecSys Design Techniques

1.4 Matrix factorization:

In this section we will deep into LFM technique which is Matrix Factorization MF by detailing its algorithm.

The basic idea behind matrix factorization is to find two matrices **P** m*K and **Q** k*n that multiplied together return a matrix **R**, which is the original matrix containing all the ratings. **P** and **Q** contain the **K** hidden features as shown in 1.5

The intuition behind this is that for a given user **u** exist a vector **pu** which measures the interest of that user **u** in the available items.

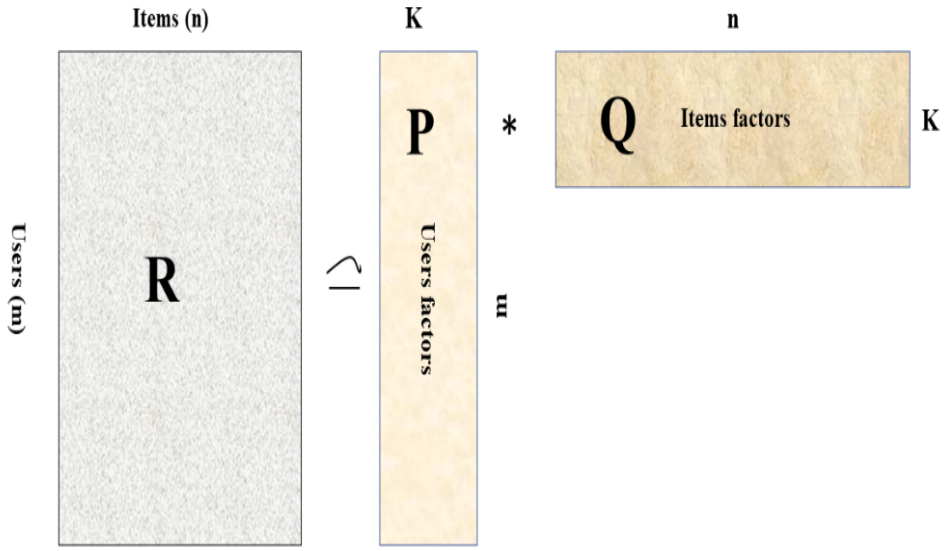


FIGURE 1.5: Matrix factorization illustration

Similarly, for an item **i** there is a vector **qi** that measures the relevance of this item amongst the users. Therefore the resulting dot product $\mathbf{qi}^T\mathbf{pu}$ captures the interaction between user **u** and the item **i** [9].

The main benefit of this method is that by capturing the overall interest of user **u** in the item **i**, we can approximate the user u's rating $\hat{\mathbf{r}}\mathbf{ui}$ towards that item i.

As introduced and explained above the main goal behind MF is to predict the ratings as follows: $\hat{\mathbf{r}}\mathbf{ui} = \mathbf{qi}^T\mathbf{pu}$.

where:

$\hat{r}ui$ = Predicted rating of user u on item i.

pu = User u factor vector.

qi = Item i factor vector.

To sum up the main challenge is to compute the mapping between users and items to the latent factors **qi** and **pu**.

During this project we adopted this approach since it is one of the state of the art techniques used in RecSys [9].

The key here is "latent factors" when you factorize a m * n matrix into two mxk and k * n matrices you are reducing your "n"items to "k"factors.

This means that instead than having your 25000 items you have say 64 factors where each factor is a linear combination of items. So maybe a factor means "a dress without buttons", another factor might mean "a dress with slit" etc. The key is that recommending based on factors is more robust than just using product similarities, maybe a user hasn't bought a sport shirt but the user might have bought other product that are related to sport like sport shows via some latent factors and those can be used.

In addition to star rating some e-commerce platform allow users to write comments in free text format, referred to as text reviews in this report. Text reviews can also be considered a type of "user ratings", although they are usually natural language texts rather than numerical values.[10]
Therefore, besides rating prediction with LFM, we will dive into the sentiment analysis of a user towards an items based on its text review written about that item after that we feed the user preferences represented in the reviews into existing CF algorithms.
The contribution of this approach are mainly two folds: Firstly, it addresses the well-known data sparseness problem in CF by allowing CF algorithms to use textual comments as an additional source of user preferences.
Secondly, it enables extending CF to domains where numerical ratings on products are difficult to collect, or where preferences on domain items are too complex to be expressed as scalar ratings[11]; like business in travel or tourism where the user leave a comment about destination, accommodation, transport ect.. For the rest of this chapter we introduce the main idea of sentiment analysis and natural language processing in recommender system.

1.5 Natural Language Processing in RecSys:

Fortunately, Natural language processing NLP and spatially ML have been used in this work. NLP and some text classification algorithms are going to be reviewed in this section.

1.5.1 Definition:

1.5.1.1 Artificial intelligence:

It's a wide-ranging branch of computer science concerned with building intelligent machines capable of performing tasks that typically require human intelligence [12].

1.5.1.2 Natural Language Processing :

It's a branch of artificial intelligence that helps computers understand, interpret and manipulate human language.
When we target M, it means we want to build systems that can learn from experience. So NLP try to build systems that can understand human language thus when we speak about ML and NLP we aim to build systems that can learn how to understand language.
The figure 1.6 explains how could AI,NLP and ML work together.

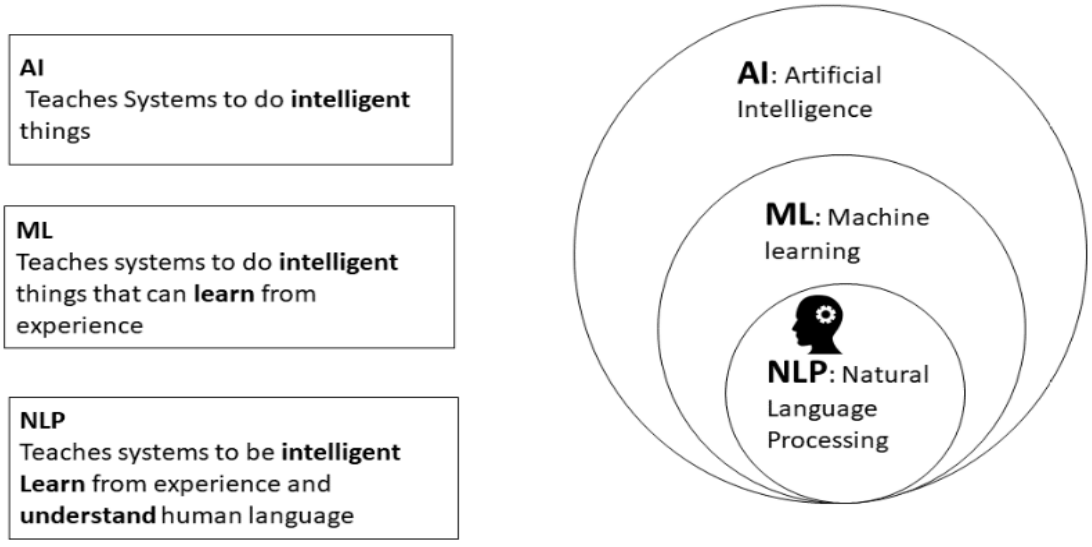


FIGURE 1.6: How AI and NLP work together

1.5.1.3 Sentiment Analysis

One of the most important fields of NLP is sentiment analysis, which is the process of discovering or mining meaningful patterns from text data.[13]
Sentiment analysis can help us attain the attitude and mood of the public which can then help us gather insightful information about the context. Related to RecSys this helps us to predict the attitude of the user u towards an item i by applying some algorithm about text classification.

1.5.2 Text Classification:

Text classification is the process of identifying text to tags or classes according to its content. It's one of the fundamental tasks in NLP with broad applications such as sentiment analysis,
The following paragraphs are about some used classifier

1.5.2.1 Naive Bayes

Naive Bayes is a family of statistical algorithms that we can make use of when doing text classification.It's based on the the Naive Bayes theorem as shown in **??** which helps us compute the conditional probabilities of occurrence of two events based on the probabilities of occurrence of each individual event[14].
This means that any vector that represents a text will have to contain information about the probabilities of appearance of the words of the text within the texts of a given category so that

the algorithm can compute the likelihood of that text's belonging to the category.

$$P(B|A) = \frac{P(B|A)P(A)}{P(B)} \tag{1.2}$$

1.5.2.2 Logistic regression

In statistics, the logistic model is a statistical model that is usually taken to apply to a binary dependent variable. The two possible dependent variable values are often labelled as "0" and "1", which represent outcomes such as pass/fail, win/lose, alive/dead or in our case in the client will be interested or not in this product. The binary logistic model is used to estimate the probability of a binary response based on one or more predictor (or independent) variables (features). It allows one to say that the presence of a risk factor increases the odds of a given outcome by a specific factor. The model itself simply models probability of output in terms of input, and does not perform statistical classification (it is not a classifier), though it can be used to make a classifier, for instance by choosing a cut-off value and classifying inputs with probability greater than the cut-off as one class, below the cut-off as the other. The logistic function known also as a sigmoid function given by the given equation 1.7

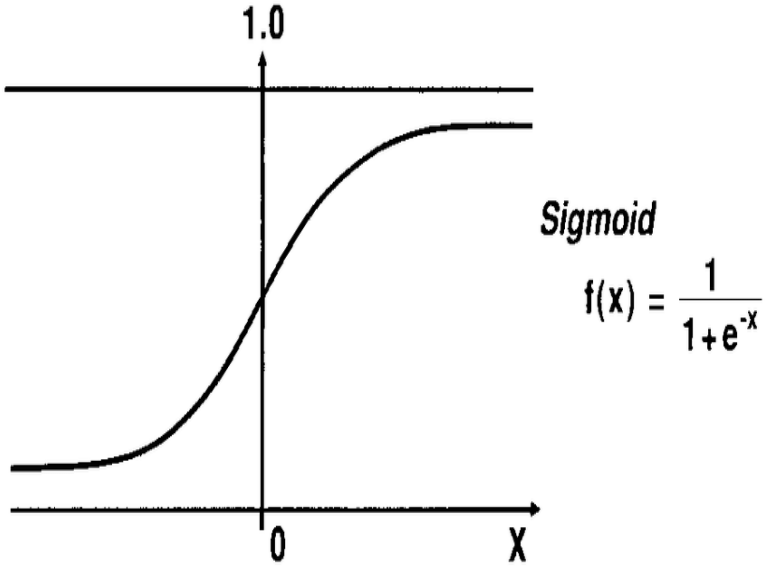


FIGURE 1.7: Sigmoid function

1.6 Goals and objectives

In this work, we suppose that we have an e-commerce website that already had his own products and visitors,
As explained above the two approaches adopted are:
• Approach 1: Use the rating to predict the preferences of users towards items: and here we will try Matrix factorization MF techniques.

• Approach 2: Using the review text to predict similar users using NLP techniques then recommend products based on the set found .
At the end we will try to combine both methods in order performance.

1.7 Conclusion

In this first chapter, we introduced the general framework of the project, present the principal theory behind recommender system.Then, we end up by mentionning the two approaches adopted with its goals.

Chapter 2

Methodology

2.1 Introduction

In this chapter we will present the data set used in this project as well as the pipeline followed by detailing the all the steps in both approaches, MF and text classification.

2.2 Pipeline of the project

As shown in 2.1 the main steps to accomplish this project are:

- Data pre-processing which is one of the most important steps in machine learning that helps in building ML models more accurately. It consists in dealing with missing values, conversion non numeric data ect..

- Exploratory Data Analysis: In this stage we explore more our data set in order to analyze it and summarize their main characteristics, often with visual methods.

- Model building with parameter optimization: This step is done for both approach's with its own features in order to predict the rating.

- Model evaluation: For both approaches we used some evaluation metrics to well evaluate our model.

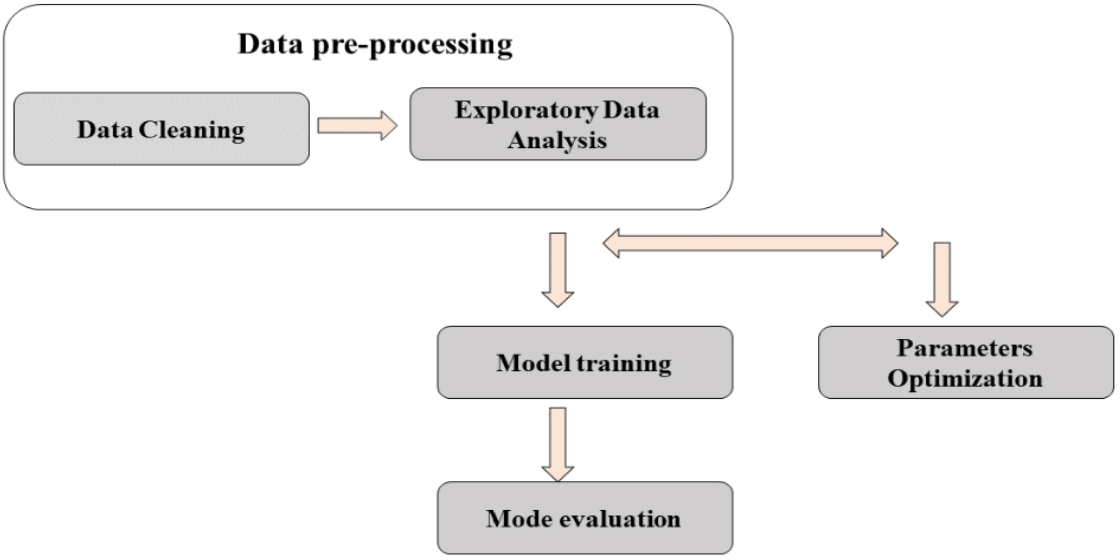


FIGURE 2.1: Pipeline of the project

2.2.1 Data-set description

We used the data set provided by Amazon which is Amazon Review Data [15] released in 2014.
It's devided into several e-commerce business area such as phones, electronics, homes and kitchen materials, fashion ect...
We've chosen as sample the later sector about fashion and it's just a simple sample and than results and models can be generalized in each domain.
As shown in figure 2.2 the Amazon fashion review data-set includes:

- Overall: rating of the product
- Verified: a boolean value that assign an item as verified or no.
- review time: time of the review (unix time)
- user Id: id of the user, e.g. A1RSDE90N6RSZF
- productId: asin, e.g. amazon.com/dp/B00006HAXW
- user name: name of the user
- review text: text of the review
- review summary: review summary
- unixReviewTime - time of the review (unix time)
- vote: helpful votes of the review
- style - a disctionary of the product metadata,
- image: gives an url of the image product

	overall	verified	reviewTime	reviewerID	asin	reviewerName	reviewText	summary	unixReviewTime	vote	style	image
0	5	True	10 20, 2014	A1D4G1SNUZWQOT	7106116521	Tracy	Exactly what I needed.	perfect replacements!!	1413763200	NaN	NaN	NaN
1	2	True	09 28, 2014	A3DDWDH9PX2YX2	7106116521	Sonja Lau	I agree with the other review, the opening is ...	I agree with the other review, the opening is ...	1411862400	3.0	NaN	NaN
2	4	False	08 25, 2014	A2MWC41EW7XL15	7106116521	Kathleen	Love these... I am going to order another pack...	My New 'Friends' !!	1408924800	NaN	NaN	NaN
3	2	True	08 24, 2014	A2UH2QQ275NV45	7106116521	Jodi Stoner	too tiny an opening	Two Stars	1408838400	NaN	NaN	NaN
4	3	False	07 27, 2014	A89F3LQADZBS5	7106116521	Alexander D.	Okay	Three Stars	1406419200	NaN	NaN	NaN

FIGURE 2.2: Data set overview

After identifying the features of the data-set we begin by discovering it, the number of each feature, the unique value and the missing values in the whole data in order to get the usefull features related to our approach.

2.2.2 Data pre-processing

The first intuition is to discover the columns of the dataframe and especially find if there is some missing values in each feature.

TABLE 2.1: Comparatif study of the data-set

Features	**Count**	**Unique**	**Missing values**
Overall: Rating	883636	5	0
ReviewerID: User ID	883636	749266	0
Asin: Product ID	883636	186189	0
ReviewText: User Review	883099 99.93%	388618	573
Review Time	883636	4608	
Vote	79900 9.04%		803736 90.95%
Image	28807 3.26%	28301	854829 96.7%
Style {Size and Color}	304569 34.46%	34309	579067 65.5%

In the table above 2.1 only the columns of overall, ReviewerID, asin and ReviewText are almost
complete with no missing value just the reviewText with percentage of missing values of 0.06% which is negligible.
After checking for NaN values, we than deleted duplicate values.
Finally we do some basic statistics like knowing the number of unique users, items and rating already given.
For the rating, it consists of numbers in the set { 1,2, 3, 4, 5}
For memory limitation we just consider the 5-core data set of amazon review fashion in which all users and items have at least **5** reviews.
In our dataset we have:

- 749232 unique users.
- 186189 unique items.
- 883636 given rating out of 749232 * 186189.
- 882402 review text written by users.
- There are 1234 users who rate a product without writing a comments about it.

2.2.3 Exploratory data analysis

Here we present and analyse some figures of data distribution.
As shown in 2.3 we noticed that the rating distribution is imbalanced with 53% as the high

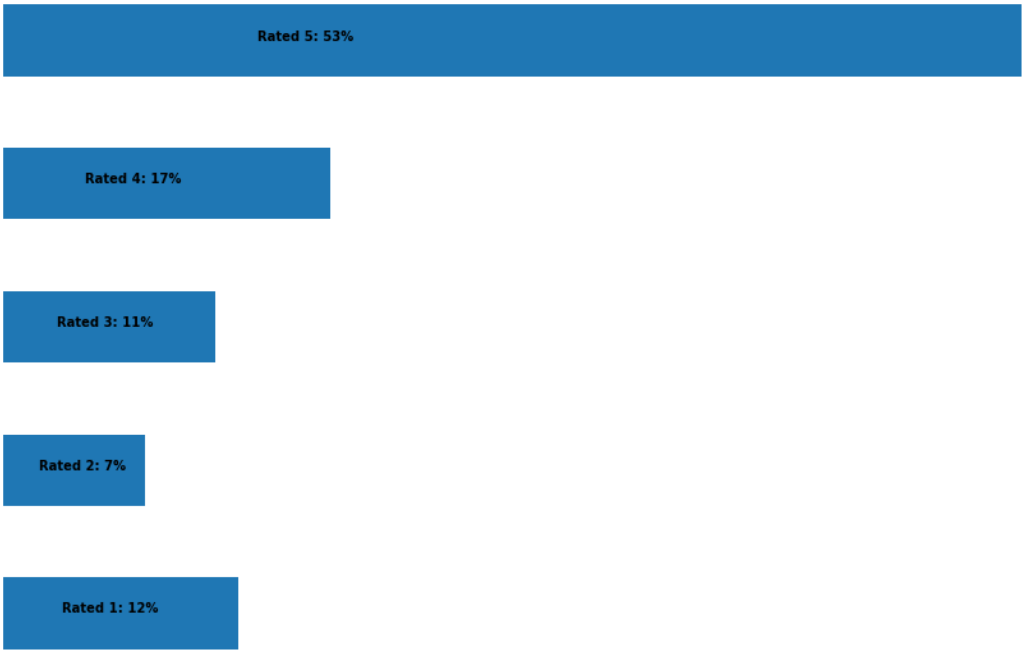


FIGURE 2.3: Rating distribution

percentage related to rating 53% and just 7% for the rating 2.
For the next figure 2.4 we grouped the rows month wise than we counted the ratings which are grouped by months and plotted them.
So, below plot shows that how many ratings are there per month.

Here we notice that the number of rating has increased during the years 2015 and 2017 and this proves indeed that the e-commerce is really very omnipresent compared to the previous years.
Moreover we tried to check the number of rating by each user in figure 2.5 by plotting the Probability Density Function PDF in figure [1]

[1] Probability density function PDF is a statistical expression that defines a probability distribution (the likelihood of an outcome) for a discrete random variable.

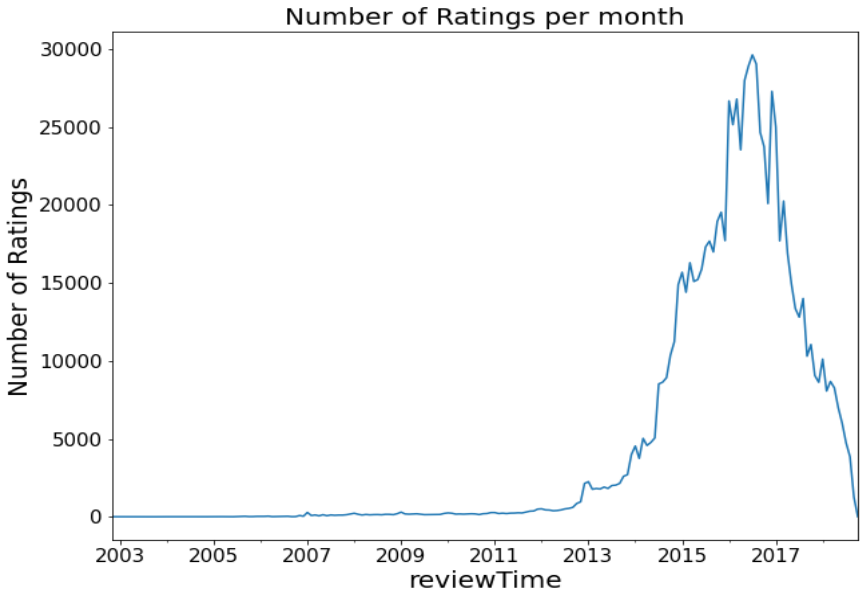


FIGURE 2.4: Number of rating per month

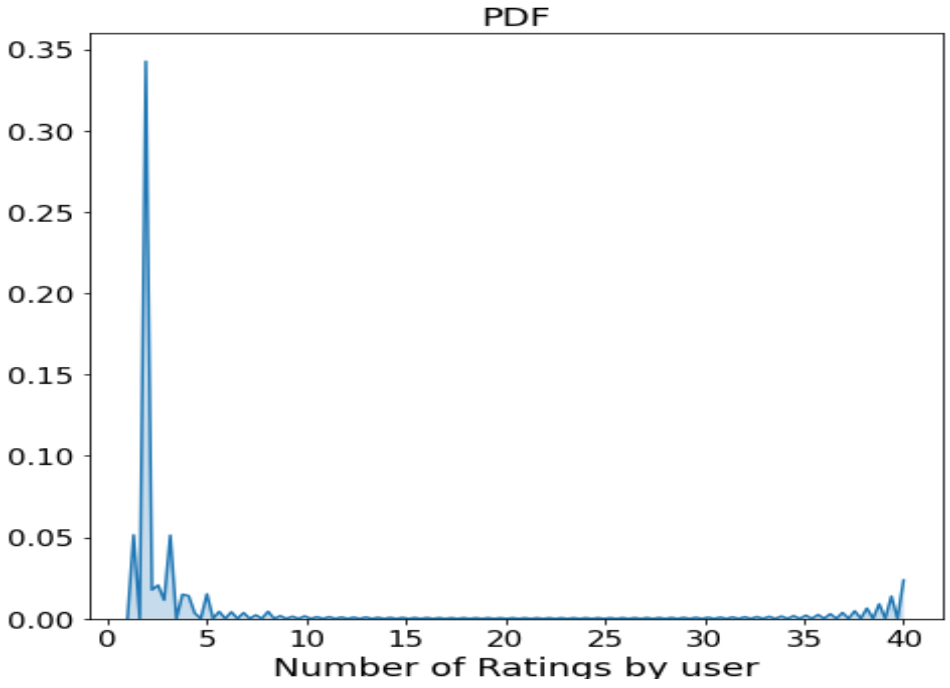


FIGURE 2.5: Probability Density Function

PDF graph shows that almost all of the users give very few ratings. There are very few users who's ratings count is high.

Related to our approaches followed and after getting information about missing values, we decided to keep just the features below:

* User ID

* item ID

* Rating
* Review Text

After discovering and understanding the structure of the data-set and with the theoretical background acquired in the first chapter plus the pipeline given in 2.1 we now build the model based on the two approaches (MF and NLP) which will be detailed in the next section.

2.3 Rating Prediction based on Matrix Factorization

The main task is to predict the rating given by user **u** to an item **i** by extracting the hidden features that express the user preferences with MF.
As explained in figure 1.5 the basic idea behind matrix factorization is to find two matrices P and Q that multiplied together return a matrix R, which is the original matrix containing all the ratings called also utility matrix or user-item matrix.

2.3.1 From utility matrix to cost function

The first step to build a matrix factorization model is to convert the data frame which contains user ID, item ID and the rating into a user-item matrix in which columns are users IDs and rows are item IDs as shown in figure 2.6.

User ID	Item ID	Rating
2	439	4
10	368	3
14	114	5
2	371	1
19	371	2



Users ID \ Items ID	114	368	371	439
2	?	?	1	4
10	?	3	?	?
14	5	?	?	?
19	?	?	2	?

FIGURE 2.6: Utility matrix illustration

As illustrated in the above figure, there are some missing entries (mentioned as a question mark) in the user-item matrix which is logic since the user doesn't rate every item.
Thus we need to discover the user preferences towards unrated items, and this preferences is modeled by the rating as a feedback.
Namely we need to predict the missing entrees to end up with a whole and complete idea

about the user interests towards each items.

To ensure this prediction we need to get features to feed the model however as mentioned in figure 2.6 we don't have present features, since we followed collaborative filtering algorithm that don't rely on content description, and here comes the power of matrix factorization that is able to extract those hidden features.

In fact each user can be described by k attributes or features : For example, feature 1 might be a number that says how much each user likes the product X.

The K factors (attributes) are saved in a user factor vector called pu from the latent matrix P

Each item (shoes) can be described by an analogous set of k attributes or features. To correspond to the above example, feature 1 for the product might be a number that says how close the shoes is to sport clothes.

Similarly to users the K attributes are saved in the factor vector qi from the matrix Q.

Than we compute the dot product between qi and pu we get the approximated value of the rating rui given by user u to item i, as illustrated in figure 2.7

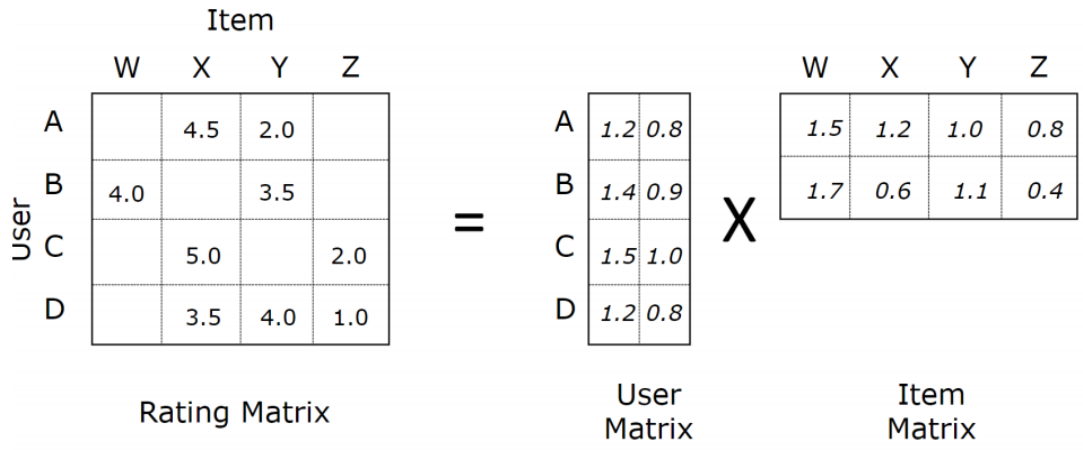


FIGURE 2.7: Latent matrix

After that we try to minimize the square difference between the actual rating and the predicted rating like the next equation 2.6

$$Minimum(P,Q) = \sum_{(u,i)} (r_{ui} - p_u q_i^{\top}) (2.1)$$

For our model to be able to generalize well and not over-fit the training set, we introduce a penalty term to our minimization equation. This is represented by a regularization factor multiplied by the square sum of the magnitudes of user and item vectors.

As a result equation 2.6 will be as follow:

$$Minimum(P,Q) = \sum_{(u,i)} (r_{ui} - p_u q_i^{\top}) + \lambda(\left\|\vec{q_i}\right\|^2 \left\|\vec{p_u}\right\|^2) \quad (2.2)$$

where pu means the uth line of matrix P, qi transpose means the ith line of matrix Q. Simply considering $\mathbf{p_u q_i}^\top$ is the predicted score to users.
The training of our model consists of minimizing the equation 2.2 which is our cost function. After an estimate of P and Q is obtained, we can predict unknown ratings by taking the dot product of the latent features for users and items.

2.3.2 Learning algorithm: Stochastic Gradient Descent

We applied Stochastic Gradient Descent SGD for minimizing the loss function because it's fast and more accurate.
The core idea of SGD is to select a rui in matrix R, then finds out the corresponding factor vector pu from the user factor matrix P and qi from the item factor matrix Q,
After that we calculates the predicted score $\mathbf{p_u q_i}^\top$, and updates parameters according to the following two rule:

$$p_u \leftarrow p_u + \gamma(e_{ui} q_i - \lambda P_u) \tag{2.3}$$

$$q_i \leftarrow q_i + \gamma(e_{ui} p_u - \lambda q_i) \tag{2.4}$$

where $e_{ui} = r_{ui} - p_u {q_i}^\top$ which represents the difference between predicted score and actual score and γ is the learning rate which controls the speed at which the model learns, or in other words, the speed at which the values of P and Q are updated to reach the minimum point of the loss function 2.6
To recap all these stuff here are the main lines of this procedure:
1/ Randomly initialize all vectors p_u and q_i, each of size 10.
2/ for a given number of times (i.e. number of epochs), repeat:

- for all known ratings r_{ui}, repeat:
- compute $\frac{\partial f_{ui}}{\partial p_u}$ and $\frac{\partial f_{ui}}{\partial q_i}$ to deduce the derivative of the squared error $e_{ui} = r_{ui} - p_u {q_i}^\top$
- update p_u and q_i with the rule presented above.

2.3.3 Implementation process

As explained in 2.1 and with the above explanation about SGD, the goal is to find p_u and q_i that minimize the loss function.

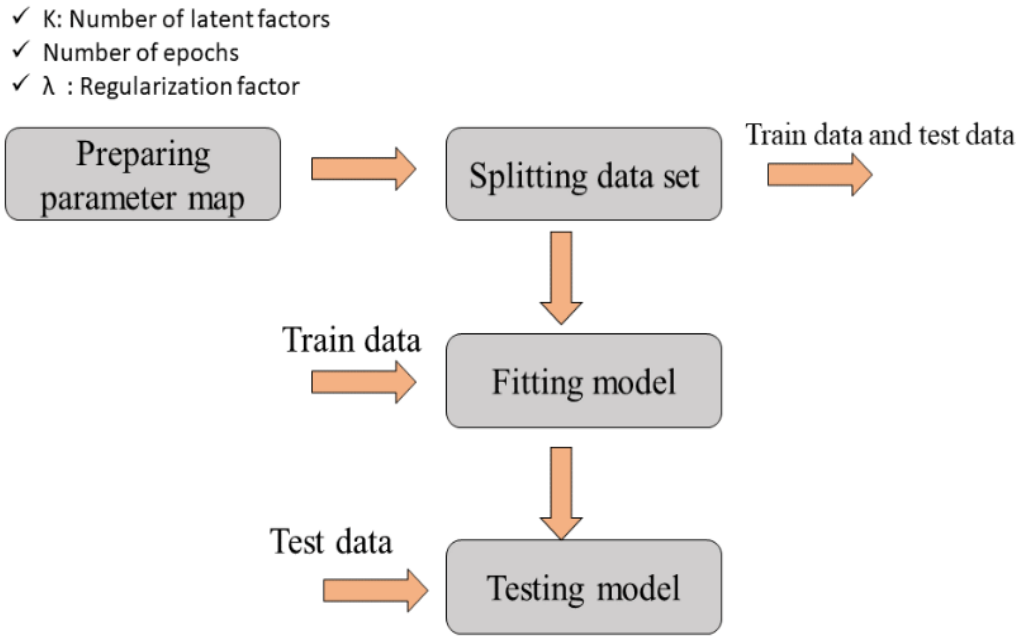


FIGURE 2.8: Pipeline of the implementation process

The figure 2.8 shows the main step of the implementation process,
which starts with the preparation of hyper parameter [2] then split the that the first step is to prepare a parameter list which consists of the hyper parameters which are:

- K: Number of latent factors
- Number of epochs (Iteration)
- λ the regularization term

To find those hyper parameter we use Grid search that is the process of performing hyper parameter tuning in order to determine the optimal values for a given model.
Suppose, a machine learning model X takes hyperparameters a1, a2 and a3. In grid searching, we first define the range of values for each of the hyperparameters a1, a2 and a3. You can think of this as an array of values for each of the hyperparameters. Now the grid search technique will construct many versions of X with all the possible combinations of hyperparameter (a1, a2 and a3) values that we defined in the first place. This range of hyperparameter values is referred to as the grid.

[2] The hyper parameter is a parameter whose value is used to control the learning process

TABLE 2.2: MF hyper parameters grid search

Hyper parameter	Definition	Range
K	Number of latent factors	[10, 20, 5, 100, 130, 150, 200]
nb epochs	Number of iterations to convert the problem	[10, 15, 30, 50, 100]
α	The learning rate: step size of each iteration	[0.001, 0.005, 0.007, 0.01, 0.05, 0.07, 0.1
λ	Regularization term to reduce overfitting	[0.01, 0.05, 0.07, 0.1, 0.2, 0.4, 0.6]

After knowing the best hyperparameters we split the data as shown in figure 2.8, into train and test set.
For the splitting we use the Leave-P-Out Cross Validation which leaves p data points out of training data, i.e. if there are n data points in the original sample then, n-p samples are used to fit the model and p points are used as the validation set it means to test the model.
This is repeated for all combinations in which original sample can be separated this way, and then the error is averaged for all trials, to give overall effectiveness.
In our case we choose p = 1 known as LOO stands for **Leave one out** cross validation. This method is generally preferred because it does not suffer from the intensive computation, since the number of possible combinations is equal to its minimum the number of data points in original sample.

2.4 Rating prediction based on text classification using Natural language Processing

Having the review text which are comments written by the user towards an item, we will try to predict the user feedback based on those comments; In fact based on text data written about items we will classify the user interests, besides that to reduce the computation cost we transform our problem from multi-class classification which has 5 classes: 1, 2, 3, 4 and 5 to binary classification with just two classes by fixing a threshold [3] here we choose 3 so that :

- If the rating is 1, 2 or 3 the class is 0.
- If the rating is 4 or 5 the label is 1.

As explained in the above figure , after predicting the interests of users towards an item with text classification, we add the prediction results to the initial data frame and convert it to obtain an unary utility matrix. In some way and compared to the above MF method it's considered as a rating with two options 1 or 0 like (like/ Dislike).

[3] Note that this threshold is fixed based on many factors like the e-commerce business area, the business model, and the situation of its revenue.

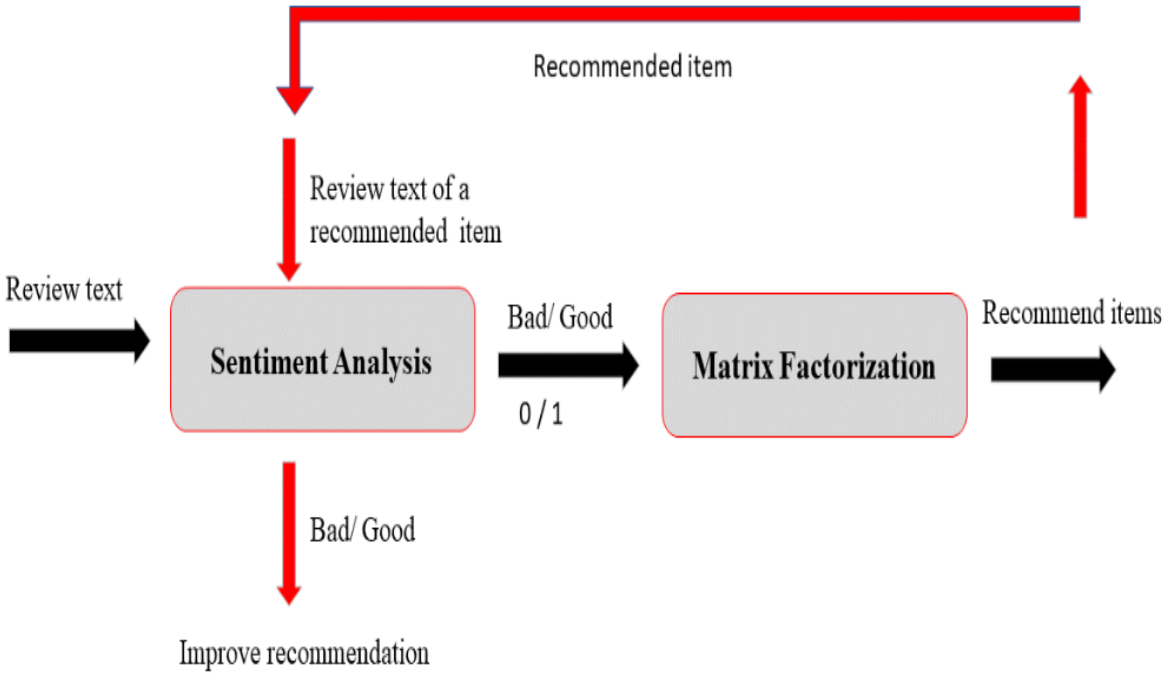


FIGURE 2.9: Why we used the sentiment analysis?

After that we implement a MF approach with the obtained dataset.
This approach can be used also to test the user interests towards one recommended items so that it helps the manager to improve its business.

Always with the same data set explained in figure 2.2 the Amazon Fashion Review data set.
we follow the general pipeline of sentiment analysis explained in figure 2.1 but with techniques related to text that are well illustrated in the next figure.

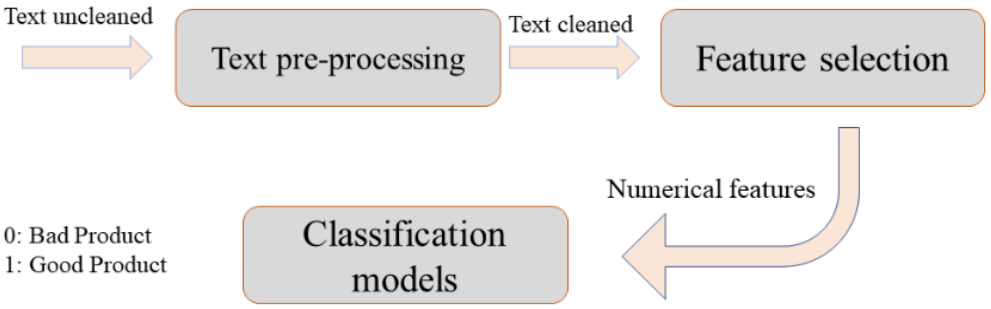


FIGURE 2.10: Text classification process

2.4.1 Text pre processing

The pre-processing of text means cleaning of noise, such as, stop words, punctuation, terms, which don't carry much weight age in context to the text.
The initial steps are:
• Escaping HTML characters: The data contains a lot of html entities like & lt; & gt; amp; which gets embedded in the original data. It is thus necessary to get rid of these entities. One approach is to directly remove them by the use of specific regular expressions.
• Deleting non alphabetic letters and punctuation: All these characters should be removed since it doesn't give any meaning in the context of sentiment analysis.
Once the data is well cleaned, we go through the following process explained with an example in figure 2.11:

- **Tokenization**: It is dividing text into small pieces (tokens) that are easier to process, generally we use space as a splitting delimiter for English language which won't be the case for other languages like for example German.

- **Remove stop words:**.Stop words: words such as "what" or "the" won't have any predicting power since they will presumably be common to all the documents. For this reason, they may represent noise that can be eliminated. We have downloaded a list of English stop words from the nltk package and then deleted them from the corpus.

- **Stemming/lemmatization:** Both terms are used to generate the root form out of a specific word, The difference is that stem might not be an actual word whereas lemma is an actual language word derived from the dictionary. For example the stem of beautiful and beauty is "beauti" which has no meaning in English dictionary by contrast to the lemma which would be "beautiful".

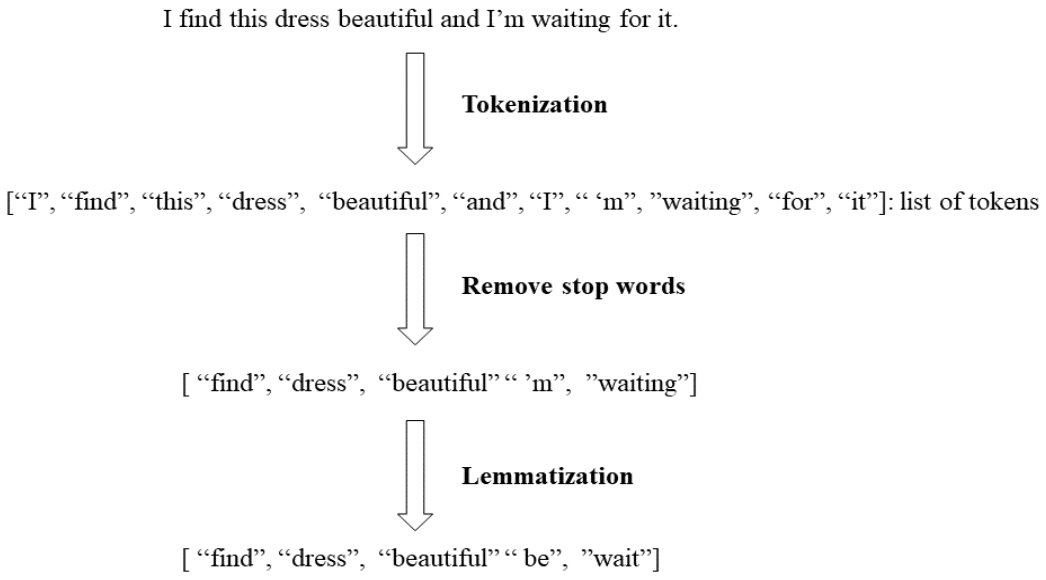


FIGURE 2.11: Text pre-processing example

2.4.2 Feature selection

Text feature selection or feature extraction is the process of taking out a list of words from the text data and then transforming them into a feature set which is usable by a classifier.

After cleaning the text we extract numerical features from it. We used the **TF-IDF** which stands for Term frequency-Inverse document frequency ,given in the next equation,it uses all the tokens in the data set as vocabulary.

The the term frequency TF is the frequency of occurrence of a token from vocabulary in each document and the number of documents in which token occurs determines the Inverse document frequency.

Here we mean by document a sentence written.

What this ensures is that,if a token occurs frequently in a document that token will have high TF but if that token occurs frequently in majority of documents then it reduces the IDF. Both these TF and IDF matrices for a particular document are multiplied and normalized to form TF-IDF of a document.

$$tfidf_{w,d} = tf_{w,d} \cdot idf_w. \tag{2.5}$$

$$idf_w = log(\frac{N}{df_w}) \tag{2.6}$$

where tf_{id}: is the number of occurrence.
df_w: number of document containing w.
N: Total number of document.

2.4.3 Classification

After getting those numerical features, we split the data into train and test data then we feed those features to the classifier to predict whether the review text is belonging to rating 1, 2, 3, 4 and 5.
We split the data into train and test set so that 80% of the data is for training and the rest 20% is for test set.
For memory limitation and computational cost we just work with 200000 out of 883636 rows.
Besides we try the Naive Bayes classifier as a baseline classifier its main idea is

- Represent a document X as a set of pairs.
- For each label y, build a probabilistic model P(X| Y = y) of documents in class y.
- To classify, select label y which is most likely to generate X:

Also we try the logistic regression with its default parameters Penalty l2: the ridge regression as a regularization technique.

2.5 Conclusion

In this chapter we discussed the methodology of this project as well as explained the pipeline and the steps in order to get results that will be presented and analyzed in the next chapter.

Chapter 3

Experimental results and discussions

3.1 Introduction

In this chapter we begin by the the implementation tools used to accomplish this work, then we present the results of the experiment and analyze it, later on we discuss how to combine them..

3.2 Implementation tools

3.2.1 Google colab

Google Colab or Colaboratory is a cloud service, offered by Google (free), based on Jupyter Notebook and intended for training and research in machine learning. This platform allows training of Machine Learning models directly in the cloud, without the need to install anything on our computer except a browser.
It offers a 12 GB RAM GPU for free, a,d it's based on jupyter notebook.



FIGURE 3.1: Google Colaboratory logo

jupyter notebook is an Open Source web application for creating and sharing documents containing code (executable directly in the document), equations, images and text. With this

application it is possible to do data processing, statistical modeling, data visualization,

3.2.2 Surprise A Python scikit for recommender systems

For matrix factorization we use Surpise a Python scikit building and analyzing recommender systems that deals with explicit rating data.
We use Surprise because it:
• Alleviate the pain of Dataset handling.
• Users can use both built-in datasets (Movielens, Jester), and their own custom datasets.
• Provide various ready-to-use prediction algorithms such as baseline algorithms, memory-based methods, model-based and many others. Also, various similarity measures (cosine, MSD, pearson...) are built-in.
• Make it easy to implement new algorithm ideas. • Provide tools to evaluate, analyse and compare the algorithms performance.
.



FIGURE 3.2: Python package logo : Surprise

3.2.3 NLTK

For sentiment analysis we used NLTK which stands for Natural Language Toolkit.NLTK it's a powerful Python package that provides a set of diverse natural languages algorithms.
NLTK consists of the most common algorithms such as tokenizing, , stemming, sentiment analysis ect.
We choose NLTK since its:
Popularity: NLTK is one of the leading platforms for dealing with language data.
• Simplicity: Provides easy-to-use APIs for a wide variety of text preprocessing methods.
• Community: It has a large and active community that supports the library and improves it.
• Open Source: Free and open-source available for Windows, Mac OSX, and Linux.

3.3 Results and discussions

3.3.1 Matrix factorization results and discussion

3.3.1.1 Evaluation metrics for MF

RMSE stands for Root Mean Squared Error is typically used to evaluate regression problems where the output (a predicted scalar value) is compared with the true scalar value output for a given data point.
It is computed as follows:

$$RMSE = \sqrt{\frac{1}{|\psi|}\sum_{i,u}(r_{ui} - \hat{r}_{ui})^2} \tag{3.1}$$

Where ψ is the set of indices of available ratings and $|\psi|$ is the cardinality of the rating dataset i.e. number of available ratings.
As the speed of prediction is also an important characteristic of a good RecSys, we also report the run time of various algorithms as a measure of their computation cost.

3.3.2 Matrix factorization results and discussion

In this subsection we first define the evaluation metrics used for text classification than we present the results.
After tuning the hyperparameters with grid searsh and based on the table 2.2 of different ranges of each hyper parameter; we find the best parameters as follow:
Number of iterations: 100
Number of latent factor K: 50
Learning rate α: 0.005
Regularization term Λ: 0.01
The two figures below presents the variation of RMSE in train and test data for multiple value of latent factor pre-defined in the table 2.2.

0.55

(a) b

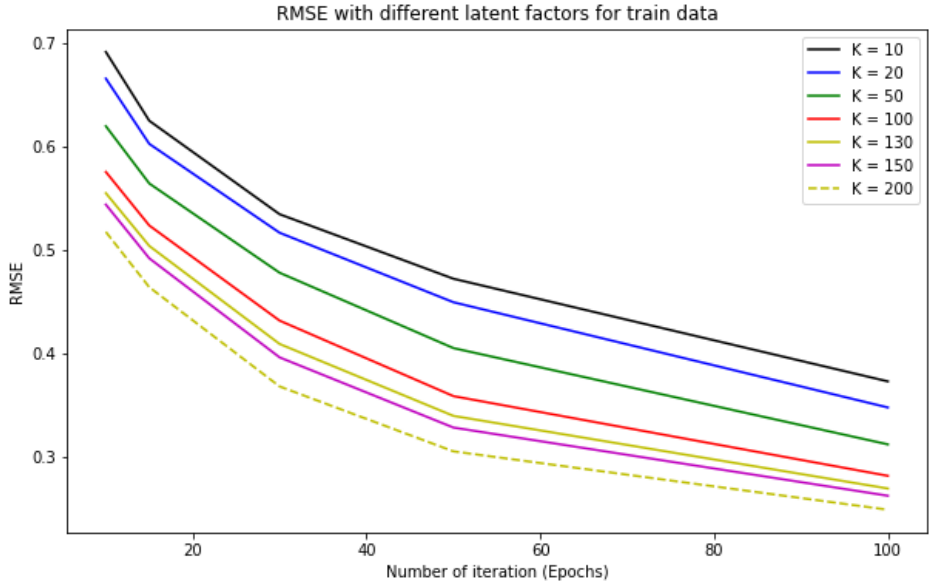


FIGURE 3.3: RMSE train data with different latent factors

0.55

(a) b

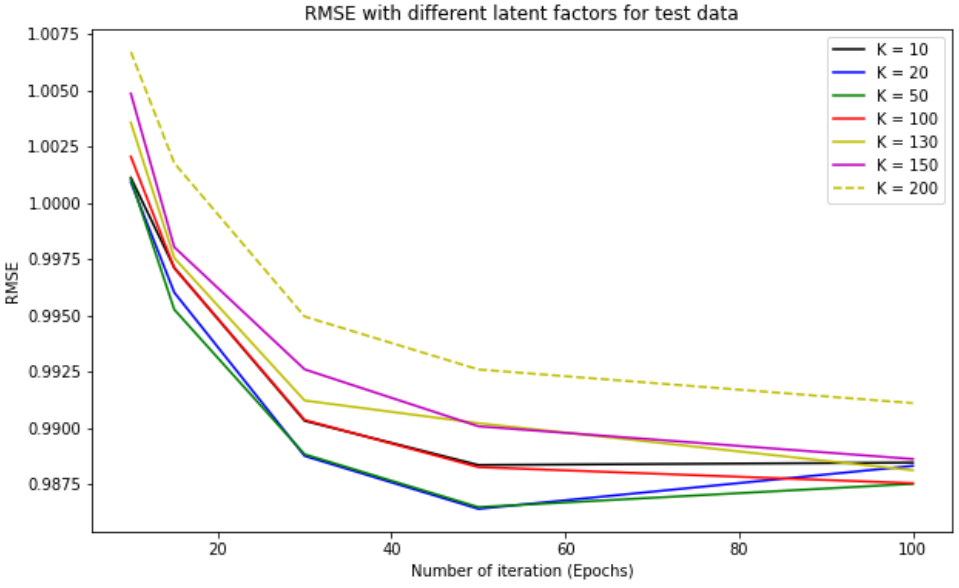


FIGURE 3.4: RMSE test data with different latent factors

As shown in figure 3.4 the best values of RMSE is 0.95 obtained with K equal to 50 and number of iteration equal to 100.
It means that the optimal number of latent features that can optimize our 2.2 is 50 with 100 epochs.
The training of our algorithm lasts about 225 ms as shown in the next figure.

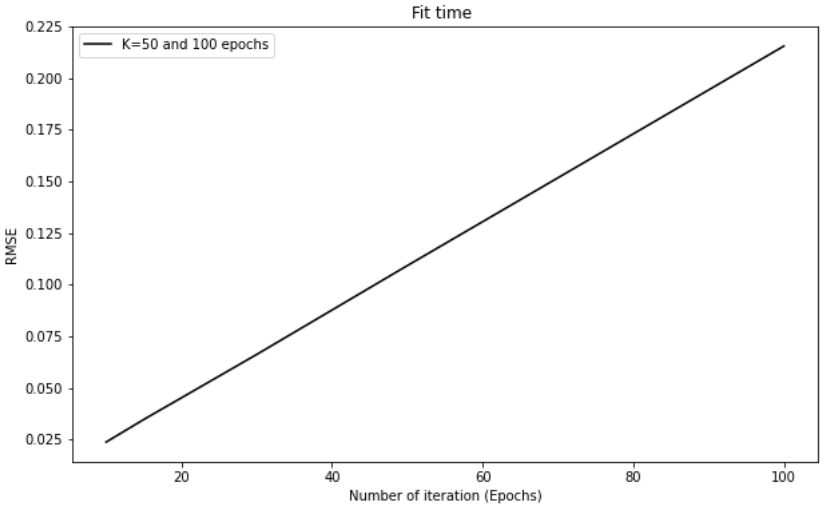


FIGURE 3.5: Training time

For the regularization factor λ we tried also several values based on 2.2, here we present the variation of RMSE with several values of λ.

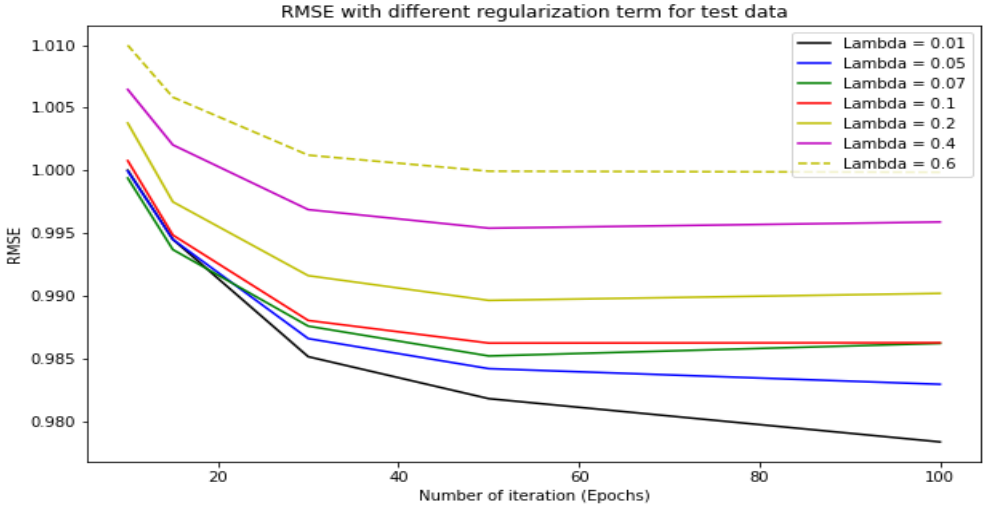


FIGURE 3.6: Variation of RMSE with several λ values

Finally we present the variation of RMSE with several values of learning rate α as shown in the next figure.

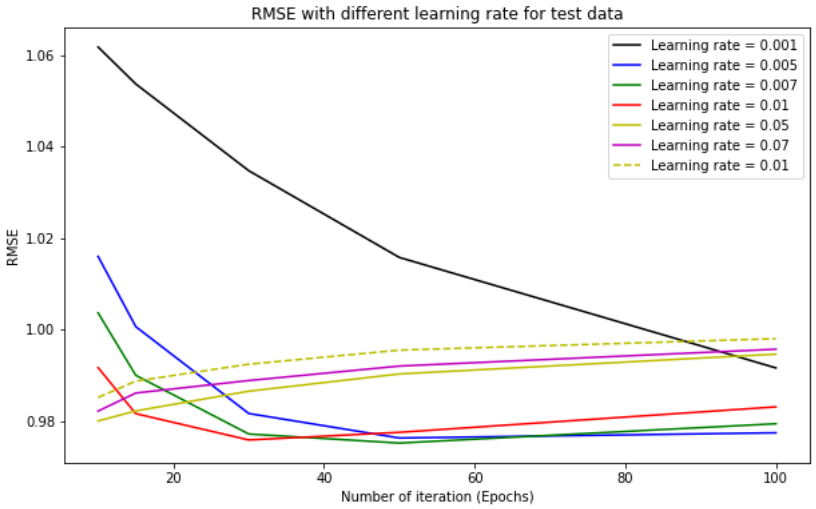


FIGURE 3.7: Variation of RMSE with several α values

Related to our data set 2.1 each user have rated just few products, with this Matrix factorization approach we could estimate the user preferences (displayed as a rating) towards every item in the data-frame. As explained before the estimation process is built after extracting the latent factors (hidden features).

By knowing those hidden features we now could complete the utility matrix explained in 2.6, we now discover those missing values (unrated items), therefore we predict what a user could rate an item in the future.

In real-life scenarios, every user wouldn't have bought all possible products. So the rating matrix will look more empty - Sparse Matrix. In this case, the model still learns to factorize the sparse matrix, but RMSE will be calculated only for the ratings that are actually present in the matrix. After obtaining the best factors to approximate the ratings we have, we then perform dot product of the factor matrices to fill in the missing entries in the rating matrix. These filled in ratings are then used to provide recommendations to the users.

After that the business manager has a complete idea about the overall preferences of each user as well as the similar preferences; Matrix Factorization exploits this similarity in user's preferences and interactions to provide recommendations.

This approach doesn't rely of any other data, every feature is present but it's hidden so this work can be applied and generalized in any other domain,

3.3.3 Sentiment analysis results and discussion

3.3.3.1 Evaluation metrics for classification

- **Confusion Matrix**It can be used as a way to visualize the results of a classification algorithm. For the binary case where 1 and 0 is the two possible options, the algorithm can be used to predict if a test sample is either 0, or 1. As a way to measure how well the algorithm performs, we can count four different metrics, here we mean by 1 positive and 0 by negative.

  1. True positive (TP), the algorithm classifies 1 where the correct class is 1.Good review text correctly identified

  2. False positive (FP), the algorithm classifies 1 where the correct class is 0. review text incorrectly classified as good feedback.

  3. True negative (TN): the algorithm classifies 0 where the correct class is 0.:review text correctly classified as bad.

  4. False negative (FN), the algorithm classifies 0 where the correct class is 1: review text incorrectly classified as bad feedback.

  The confusion matrix is simply these four values visualized in one table, see Figure 3.8 .

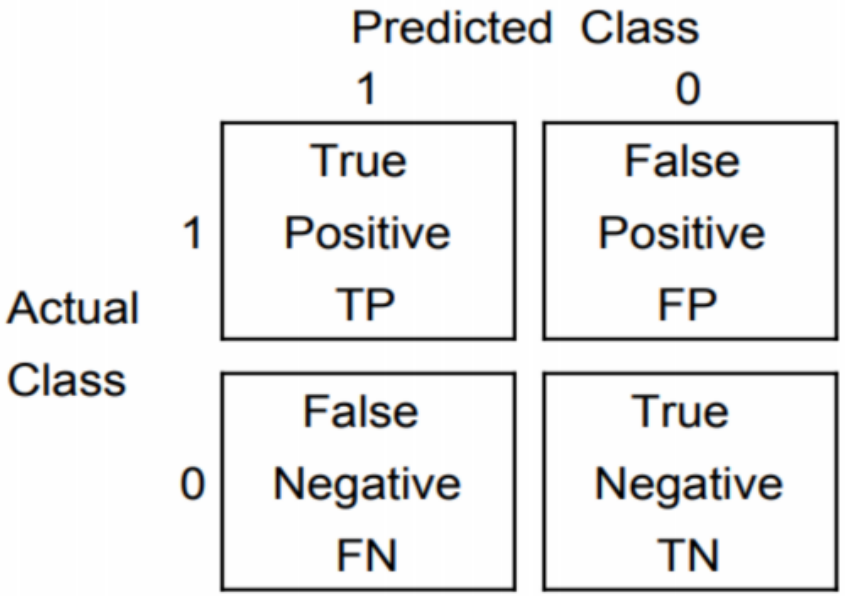


FIGURE 3.8: Confusion matrix

- **Precision and Recall**As a way to evaluate the performance of an machine learning algorithm, one can use precision and recall. Precision is defined as:

$$precision = \frac{TP}{TP + FP} \tag{3.2}$$

(3.3)

And recall defined as the number of correct results divided by the number of results that should have been returned

$$recall = \frac{TP}{TP + FN} \tag{3.4}$$

While recall expresses the ability to find all relevant instances in a dataset, precision expresses the proportion of the data points our model says was relevant actually were relevant.
A very high precision means that the algorithm classifies almost no inputs as positive unless they are positive. A high recall would mean that the algorithm misses almost no positive values.

- **F1-score**is the harmonic mean of precision and recall. The f-score is defined as:

$$F1 - score = 2 * \frac{Precision * recall}{Precision + recall} \tag{3.5}$$

- **Accuracy** is the proportion of true results among the total number of cases examined; it's defined as:

$$accuraccy = \frac{TP + TN}{TP + FP + FN + TN} \tag{3.6}$$

3.3.3.2 Results of text classification

Here we present the result obtained after training the data;

TABLE 3.1: Result table

Model	Accuracy	Class	Precision	recall	F1 Score	Support
Logistic regression	0.901	**Negative feedback** **Positive feedback** **Average**	0.819 0.909 0.895	0.45 0.98 0.901	0.581 0.944 0.889	7561 42332 49693
Naive Bayse	0.860	**Negative feedback** **Positive feedback** **Average**	0.934 0.864 0.875	0.126 0.998 0.865	0.222 0.9285 0.820	7561 42332 49893

As presented in the table below the algorithm of logistic regression gives us the better result with a percentage of accuracy almost 90% and a F1 score in average 88,9%.
This is expected since the Naive Bayes also assumes that the features are conditionally independent but real data sets are never perfectly independent they can be close.
To well visualize its result we present the confusion matrix in the next figure.

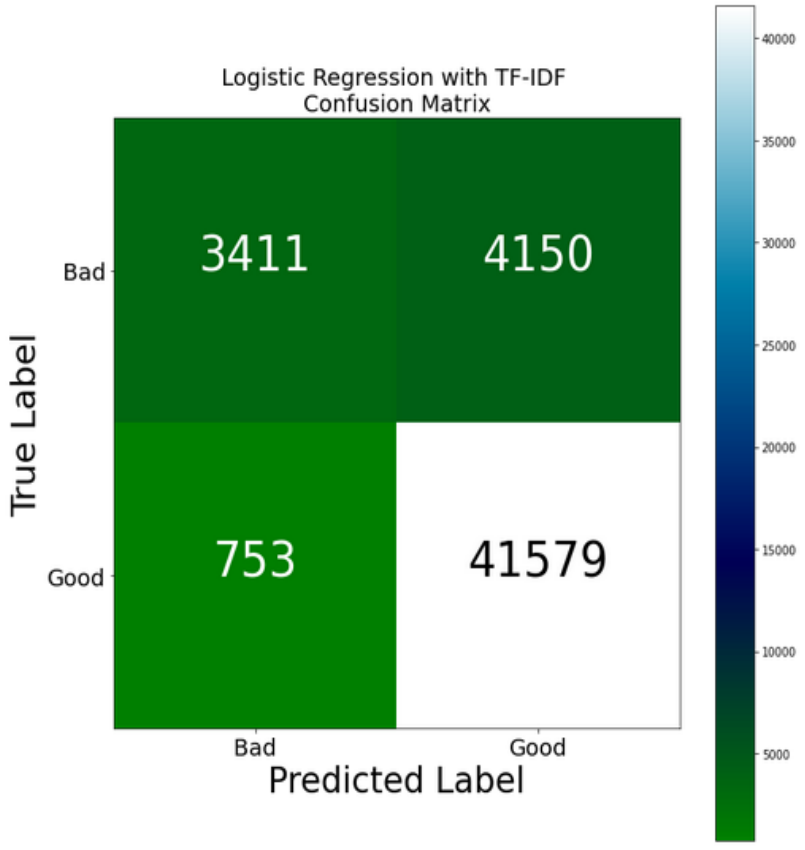


FIGURE 3.9: Confusion matrix results

From the confusion matrix we can deduce that there are 3411 review text labeled as bad review and was predicted correctly as negative feedback.
Also we see the FN part; there are 41579 review text labeled as good comments and was correctly classified as negative.
The model with logistic regression gave accepted results, in addition this can be performed by trying a to build a model based on deep learning [1]for classification but this needs more data to reach good results.

The sentiment analysis here is used to give correct feedback in order to feed it to the matrix factorization model because a user rating may not project the real interests, in fact he can be quiet pessimistic or optimistic so that loose the relevance of the rating. That's why generally a review text written could be well representative.

In addition the sentiment analysis is used to test the feedback of the user towards a recommended item by analyzing the review text and predicting the class of the sentiment as good or bad item.

3.4 Conclusion

In this chapter we presented and analyzed the results, then we shed the light on the importance of the sentiment analysis in recommender system.

[1] Deep learning is a sub-field of machine learning dealing with algorithms inspired by the structure and function of the brain called artificial neural

Conclusion and Future Perspectives

A recommender system is an Information Retrieval technology that improves access and proactively recommends relevant items to users by considering the users' explicitly mentioned preferences and objective behaviors. Therefore, it is necessary to build high quality recommender systems.
In this context is done this intern in which we focused on developing recommendations strategies to address several pertinent issues, relevant to RecSys design.
1. Our designs are built on the principles of latent factor models for collaborative filtering; We used explicit rating information as it is more reliable than implicit data.
2. We detailed how to convert our problem from recommendation problem to an optimization one that aim to predict rating.
3. we present a pipeline to use the review text in order to boost recommended items by using some algorithm for classification.
The structure of this work is explained and divided into 3 chapters:
The first chapter of this report, shed the light on the needed notions about recommender system its different techniques besides the need of sentiment analysis in recommendation.
The second chapter in charge of the system modeling and design, took care of identifying the project challenges and their corresponding suggested solutions. It highlighted the high-level design and the machine learning pipelines for both Matrix factorization and sentiment analysis.
The third chapter presented the used technologies in this work. It also detailed the results and explained it

Latent factor model as described before as the state of the art of recommender system technique. The hidden features extracted with LFM are used to recommend items to customers giving the most performant results but when speaking to the new field or research which is explainable recommendation how could the model be suitable to explain well the causes of a recommendation with latent factor model?

Bibliography

[1] The ubiquitousness of E-commerce . URL https://www.shopify.com/encyclopedia/what-is-ecommerce/.

[2] CAGR Definition and statistics. URL Source:https://www.industryarc.com/Research/Recommendation-Engine-Market-Research-50099/.

[3] Iacovou N. Suchak M.-Bergstrom P. Riedl J. (1994 October) Resnick, P. GroupLens: an open architecture for collaborative filtering of netnews. URL http://ccs.mit.edu/papers/CCSWP165.html/.

[4] Lanning S. (2007-August) Bennett, J. The netflix prize. In Proceedings of KDD cup and workshop (Vol. 2007, p. 35). URL https://www.cs.uic.edu/~liub/KDD-cup-2007/NetflixPrize-description.pdf/.

[5] Sofia Serrano Jinny Cho, Ryan Gorey. Book Recommendation System. URL http://cs.carleton.edu/cs_comps/1617/book_rec/final-results/paper.pdf/.

[6] XuLexi XuShow Yu Liu, Jingwen NieLexi. Clothing Recommendation System Based on Advanced User-Based Collaborative Filtering Algorithm. URL https://www.researchgate.net/publication/321916529_Clothing_Recommendation_System_Based_on_Advanced_User-Based_Collaborative_Filtering_Algorithm/.

[7] Collaborative Filtering vs. Content-Based Filtering:differences and similarities. URL https://arxiv.org/pdf/1912.08932.pdf.

[8] Sutheera Puntheeranurak To Thi Thuan. Hybrid recommender system with review helpfulness features. URL https://ieeexplore.ieee.org/abstract/document/7022397/.

[9] Robert Bell Yehuda Koren and Chris Volinsky. Matrix Factorization Techniques for Recommender Systems IEEE 2009. URL https://ieeexplore.ieee.org/document/5197422/.

[10] Rafael Martins. A collaborative filtering approach based on user's reviews. URL https://core.ac.uk/download/pdf/37524167.pdf.

[11] Integrating Collaborative Filtering and Sentiment Analysis: A Rating Inference Approach. URL http://www.mysmu.edu/staff/caneleung/pub/rsw06_ratingInfer.pdf.

[12] Stephen Chi-fai Chan Cane Wing-ki Leung. A Brief Introduction To Artificial Intelligence. URL https://ieeexplore.ieee.org/document/1152096.

[13] Pushpak Bhattacharyya. Sentiment Analysis. URL https://ieeexplore.ieee.org/document/6691379/.

[14] Haiyi Zhang ;. Naïve Bayes Text Classifier. URL https://ieeexplore.ieee.org/document/4403192.

[15] Julian McAuley Jianmo Ni, Jiacheng Li. Amazon Review Data. URL https://nijianmo.github.io/amazon/index.html/.